

Brief de projet — Segmentation et classification des clients (Credit Card Customer Data)

Contexte du projet

Vous êtes développeur IA junior au sein d'une équipe data d'un établissement financier souhaitant mettre en place un système intelligent de **segmentation client** à partir du comportement d'utilisation de carte de crédit (solde, achats, avances de cash, paiements, etc.).

L'objectif est double :

1. **Découvrir**, via des techniques non supervisées, des profils de comportement homogènes parmi les clients (segmentation).
2. **Construire un modèle supervisé** capable de prédire automatiquement le segment d'un nouveau client, puis de **déployer** ce modèle avec suivi d'expérimentation (MLflow), orchestration (Airflow) et interface utilisateur (Streamlit).

Le jeu de données contient environ 9000 clients actifs, décrits par 7 variables comportementales sur 6 mois.

Dictionnaire des variables

Variable	Description
CUST_ID	Identifiant unique du client
BALANCE	Solde restant sur le compte, disponible pour effectuer des achats
PURCHASES	Montant total des achats effectués par le client
ONEOFF_PURCHASES	Montant du plus gros achat réalisé en une seule fois (achat ponctuel)
INSTALLMENTS_PURCHASES	Montant des achats réglés en plusieurs fois (paiement échelonné)
CASH_ADVANCE	Montant des avances de trésorerie (retraits de cash sur la carte)
CREDIT_LIMIT	Plafond de crédit accordé au client
PAYMENTS	Montant total des paiements effectués par le client sur la période

Feature Story 1 : Analyse exploratoire des données (EDA)

Objectif : comprendre la structure et la qualité des données avant tout traitement.

Tâches :

- Importer les données, vérifier types, dimensions et aperçu général.
- Identifier valeurs manquantes et doublons.
- Analyser la distribution des variables numériques : histogrammes + skewness affiché, pour repérer les variables asymétriques.
- Étudier les corrélations entre variables (heatmap de la matrice de corrélation).
- Repérer les valeurs aberrantes (outliers) sur chaque variable numérique.

Visuels à produire :

- Histogrammes (+ courbe de densité KDE) de chaque variable numérique, avec le coefficient de skewness affiché en titre, pour repérer d'un coup d'œil les variables asymétriques.
- Heatmap de la matrice de corrélation, pour visualiser les liens entre les features.
- Boxplots (boîtes à moustaches) de chaque variable numérique, pour repérer visuellement les outliers (bornes IQR).
- Tableau récapitulatif des valeurs manquantes (nombre + %) par colonne, trié par ordre décroissant.

Feature Story 2 : Prétraitement des données

Objectif : produire un dataset nettoyé de référence, puis une branche séparée dédiée au clustering — sans jamais mélanger les deux, pour ne pas polluer la classification (Feature Story 5) avec des transformations propres au clustering.

Tâches :

- **Nettoyage minimal** (`df_clean`) : suppression des doublons, retrait des colonnes inutiles, Sauvegarder `df_clean` avant toute transformation : c'est le point de départ commun au clustering et, plus tard, à la classification.
- **Préparation clustering** (sur une copie dédiée, ex. `df_prepare_clustering`, jamais sur `df_clean` directement) :
 1. Transformation logarithmique pour corriger l'asymétrie des 7 variables (montants positifs, distribution étalée à droite).
 2. Standardisation (`StandardScaler`), toujours **après** le log — jamais l'inverse.
 3. Réduction de dimension (PCA) : réaliser une PCA complète, analyser la variance expliquée par composante et la variance cumulée, puis localiser le nombre minimal de composantes qui permet de dépasser un seuil de 80 % de variance cumulée — ce nombre sert de première estimation, affinée ensuite conjointement avec `k` en Feature Story 3.

Visuels à produire :

- Histogrammes après transformation logarithmique, pour vérifier visuellement la correction de l'asymétrie (skew qui se rapproche de 0).
- Tableau avant/après comparant le skewness de chaque variable, pour objectiver l'amélioration apportée par la transformation.
- Courbe de variance expliquée cumulée de la PCA, avec seuil à 80 % marqué, pour visualiser le nombre de composantes retenu en première estimation.

Feature Story 3 : Entraînement des modèles de clustering (K-means et DBSCAN)

Objectif : déterminer conjointement le nombre de composantes PCA et le nombre de clusters, sans les fixer a priori, via une recherche en grille.

Tâches :

- **Recherche en grille K-means** : sur les données transformées et réduites par PCA (`df_prepare_clustering`), tester plusieurs nombres de composantes (ex. 2 à 5) croisés avec plusieurs

k (ex. 2 à 7), en calculant à chaque combinaison inertie et silhouette. Synthétiser en heatmap (composantes × **k**, valeur = silhouette), compléter par une courbe elbow et une courbe silhouette pour confirmer le choix de **k**. Vérifier l'équilibre des clusters (aucune classe < ~10 %) avant validation. Entraîner le modèle retenu sur **df_prepare_clustering**, puis rattacher le résultat à **df_clean** sous forme de colonne **cluster_kmeans**.

- **Recherche en grille DBSCAN** : même logique, avec plusieurs couples (**eps**, **min_samples**) croisés aux composantes ; calculer silhouette, nombre de clusters et proportion de bruit ; synthétiser dans un tableau trié par silhouette décroissant. Entraîner avec la meilleure combinaison, rattacher le résultat à **df_clean** sous forme de colonne **cluster_dbscan** (-1 = bruit), et ajouter une colonne booléenne **est_atypique_dbscan**.
 - **Silhouette le plus élevé** entre les deux méthodes → indique la meilleure séparation des clusters.
 - **Couverture des points** : DBSCAN classe une partie des clients en bruit (-1), donc s'il reste une proportion significative de bruit, il ne couvre pas tous les clients — problématique puisque **target** doit être définie pour **tous** les clients, sans valeur manquante.
 - **Équilibre des clusters** : vérifier qu'aucune méthode ne produit un cluster écrasant (> 90 %) ou trop petit (< 10 %).
 - **En pratique, K-means est recommandé par défaut** dès lors que tous les points doivent être classés (nécessaire pour **target**) — DBSCAN n'est retenu à sa place que s'il obtient un silhouette nettement supérieur **et** une proportion de bruit très faible (ex. < 5 %).
 - Une fois la méthode choisie, créer une colonne **cluster_final** dans **df_clean** = copie de la colonne de la méthode retenue (ex. **df_clean["cluster_final"] = df_clean["cluster_kmeans"]**) : c'est cette colonne unique qui sera utilisée en Feature Story 4, sans ambiguïté sur la méthode choisie.

Point de vigilance : l'entraînement (fit) se fait toujours sur **df_prepare_clustering** — jamais sur **df_clean**, qui reste en valeurs originales. Seules les colonnes de labels (**cluster_kmeans**, **cluster_dbscan**, **est_atypique_dbscan**, **cluster_final**) sont ramenées sur **df_clean**.

Visuels à produire :

- Heatmap composantes PCA × **k**, silhouette en valeur.
- Courbe elbow et courbe silhouette (vs **k**).
- Tableau des meilleures combinaisons DBSCAN, trié par silhouette décroissant.
- Tableau comparatif K-means vs DBSCAN (silhouette, Davies-Bouldin, Calinski-Harabasz, % de bruit).
- Nuage de points final (PCA=2, **k** retenu), coloré par **cluster_final**.
- Superposition des centroïdes K-means sur ce nuage de points, pour visualiser le centre de chaque cluster dans l'espace PCA.

Feature Story 4 : Analyse et interprétation des clusters

Objectif : donner un sens aux numéros de cluster (0, 1, 2...) — comprendre qui sont vraiment ces clients, pour pouvoir en parler avec des mots simples.

Tâches :

- **Calculer les moyennes par cluster (vue standardisée)** : pour chaque cluster, calculer la moyenne de chaque variable sur le dataset transformé/standardisé. Ça permet de comparer facilement les clusters

entre eux, sur une même échelle, et de repérer en un coup d'œil les variables les plus extrêmes (positives ou négatives) pour chaque cluster — ce sont elles qui racontent le profil du cluster.

- **Calculer les mêmes moyennes par cluster (vue réelle)** : refaire le même calcul, mais sur `df_cleaned` (les vraies valeurs, non standardisées, avec la même colonne `cluster_kmeans` déjà rattachée). Ce deuxième tableau donne les montants réels, indispensables pour écrire des seuils compréhensibles par le métier.

Exemple de conditions obtenues à partir des deux tableaux (chiffres à titre illustratif, dépendent de vos propres résultats) :

Cluster	Condition retenue
0	<code>PURCHASES > 2000</code> et <code>PAYMENTS > 3000</code> et <code>CREDIT_LIMIT > 6000</code>
1	<code>CASH_ADVANCE < 100</code> et <code>CREDIT_LIMIT < 3000</code> et <code>PAYMENTS < 700</code>
2	<code>CASH_ADVANCE > 1500</code> et <code>PURCHASES < 100</code>

Bien vérifier aussi les clusters qui se ressemblent sur une variable mais divergent sur une autre.

- **Donner un nom à chaque cluster** : à partir des conditions ci-dessus, choisir un nom clair et cohérent pour chaque cluster (ex. cluster 0 → "Clients Premium" ; cluster 1 → "Clients à Faible Activité" ; cluster 2 → "Clients à Risque"), en expliquant chaque nom en 2-3 phrases décrivant le comportement type du segment.
- **Créer la colonne finale** : établir la correspondance entre chaque numéro de cluster et le nom métier retenu, puis l'appliquer à `df_cleaned` pour créer la colonne `target`, en vérifiant qu'aucun numéro de cluster n'a été oublié, qu'aucune valeur n'est manquante dans `target`, et que ses proportions correspondent bien à celles des clusters d'origine.

Point de vigilance : le numéro attribué à chaque cluster par K-means (0, 1, 2...) n'a pas de signification fixe — il peut changer d'un entraînement à l'autre, même avec les mêmes paramètres. Toujours reconstruire les deux tableaux de moyennes (standardisé + réel) sur son propre résultat avant d'attribuer les noms et les seuils, plutôt que de réutiliser ceux d'un autre run.

Visuels à produire :

- Heatmap des moyennes par cluster (vue standardisée), pour voir en un coup d'œil ce qui différencie chaque groupe.
- Diagramme en barres avec la taille de chaque cluster (nombre de clients et %).
- Petits graphiques en barres comparant les clusters sur les variables clés (`CASH_ADVANCE`, `PURCHASES`, `PAYMENTS`), en vraies valeurs.

Feature Story 5 : Classification supervisée et évaluation des modèles

Objectif : prédire automatiquement le segment (`target`) d'un nouveau client.

Tâches :

- **Point de départ** : `df_clean` + `target`, nettoyé mais **pas** transformé/standardisé/réduit par PCA (ces traitements sont propres au clustering et créeraient une fuite de données pour la classification).

- Définir **X** (variables comportementales) et **y** (**target**), en excluant **target** et toute colonne de clustering (**cluster_kmeans**, **cluster_dbscan**, etc.).
- Split train/test stratifié, gestion du déséquilibre des classes si besoin (sur/sous-échantillonnage via **imblearn**).
- **Construire un pipeline Scikit-learn unique** (**Pipeline**([("scaler", **StandardScaler**()), ("classif", ...)])), regroupant le prétraitement et le modèle en un seul objet, pour garantir que les mêmes étapes sont appliquées à l'entraînement et à la prédiction.
- Entraîner plusieurs versions de ce pipeline, en changeant uniquement l'étape **classif** : Random Forest, SVM, Decision Tree, Régression Logistique.
- Évaluer chaque pipeline (matrice de confusion, précision, rappel, F1-score), valider par cross-validation, optimiser via **GridSearchCV/RandomizedSearchCV** appliqué directement sur le pipeline complet — le prétraitement doit être ré-appris (**fit**) à chaque fold, pas une seule fois en amont.
- Comparer les modèles et sélectionner le meilleur.
- **Point de vigilance essentiel** : le prétraitement (standardisation, etc.) doit être appris uniquement sur **X_train**, à l'intérieur du pipeline, puis appliqué à **X_test** — jamais l'inverse, sous peine de fausser l'évaluation.
- Sauvegarder le pipeline complet (prétraitement + modèle) via **joblib** : c'est cet objet unique qui sera rechargé par Streamlit.

Visuels à produire :

- Matrice de confusion (heatmap annotée) pour chaque modèle testé, pour visualiser immédiatement les classes bien/mal prédites.
- Tableau comparatif des modèles (accuracy, précision, rappel, F1-score, temps d'entraînement), trié par la métrique la plus pertinente pour le cas d'usage.
- Diagramme en barres comparant le F1-score (ou l'accuracy) de tous les modèles testés, pour identifier visuellement le meilleur en un coup d'œil.
- Graphique d'importance des variables (**feature_importances_** pour Random Forest, ou coefficients pour la Régression Logistique), pour identifier les variables les plus déterminantes dans la prédiction du segment.
- Courbes de validation croisée (boxplot des scores obtenus sur chaque fold), pour visualiser la stabilité et la robustesse du modèle retenu.

Feature Story 6 : Suivi des expérimentations avec MLflow

Objectif : tracker chaque run d'entraînement de classification, pour comparer objectivement les configurations testées et garder une trace de ce qui a été essayé (initiation, centrée sur la classification uniquement — le clustering n'est pas tracké à ce stade).

Tâches :

- **Initialiser l'expérimentation** : créer un espace de suivi dédié avec **mlflow.set_experiment("classification_clients")**, pour regrouper tous les runs de ce projet au même endroit.
- **Démarrer un run par entraînement** : englober chaque entraînement de modèle dans un bloc **with mlflow.start_run()**, pour que MLflow capture automatiquement tout ce qui s'y passe.
- **Logger les hyperparamètres testés** (**mlflow.log_param**) : par exemple le type de modèle, la profondeur d'un arbre, le nombre d'estimateurs — tout paramètre qui varie d'un run à l'autre et qui

explique une différence de résultat.

- **Logger les métriques obtenues** (`mlflow.log_metric`) : accuracy, précision, rappel, F1-score — une valeur par métrique et par run, pour pouvoir ensuite les trier/comparer dans l'interface.
- **Logger le pipeline entraîné** (`mlflow.sklearn.log_model`) : sauvegarder l'objet pipeline complet (prétraitement + modèle) directement dans MLflow, pas seulement les métriques — ça permet de récupérer et réutiliser n'importe quel run plus tard, sans avoir à le ré-entraîner.
- **Logger des artefacts complémentaires** : la matrice de confusion (image) et/ou le rapport de classification complet (texte), via `mlflow.log_artifact`, pour garder une trace visuelle du comportement du modèle, pas seulement un score.
- **Comparer les runs visuellement** : lancer l'interface (`mlflow ui`), qui affiche tous les runs sous forme de tableau triable — utile pour repérer en un coup d'œil la meilleure configuration selon la métrique choisie (ex. trier par F1-score décroissant).

Visuels à produire :

- Capture d'écran (ou export) du tableau comparatif des runs dans l'interface `mlflow ui`, trié par la métrique retenue.
- Matrice de confusion loggée comme artefact pour le run du meilleur modèle.

Feature Story 7 : Application Streamlit (restitution / démonstration)

Objectif : démontrer le modèle en conditions réelles.

Tâches :

- Développer une app permettant de saisir/charger un profil client, prédire son segment (`target`) et afficher le résultat (nom du segment, probabilités par classe si disponibles).
- Charger **exclusivement** le pipeline scikit-learn unique sauvegardé en Feature Story 5 — jamais le modèle seul, et ne jamais recoder manuellement le prétraitement dans Streamlit (un simple `pipeline.predict(nouveau_client_brut)` suffit).

Feature Story 8 : Documentation et reproductibilité

Objectif : garantir que le projet soit compréhensible et reproductible par un tiers.

Tâches :

- Commenter le code (cellules Markdown si notebook), en particulier les décisions non triviales (choix de `k`, `eps`, dataset d'entrée pour la classification, seuils métier).
- Séparer le code en modules réutilisables (`extraction.py`, `preprocessing.py`, `clustering.py`, `classification.py`, `pipeline.py` (bonus)) plutôt qu'un notebook monolithique.
- Fournir un `README.md` : description du projet, structure du dépôt, installation, exécution du pipeline, lancement de MLflow/Streamlit.
- Planifier les tâches dans Jira (Epics + tickets détaillés, estimation de charge).

Structure du projet

```
projet-segmentation-clients/
├── data/
│   ├── raw/                # dataset original
│   └── processed/          # df_clean, df_prepare_clustering
├── notebooks/              # exploration, prototypage
├── src/
│   ├── preprocessing.py
│   ├── clustering.py
│   ├── classification.py
│   └── pipeline.py
├── models/                  # pipelines sérialisés (.joblib)
├── app/
│   └── app.py               # application Streamlit
├── dags/                    # DAG Airflow (bonus)
├── mlruns/                  # suivi MLflow
├── pyproject.toml           # dépendances et config du projet (uv)
├── uv.lock                  # versions figées, garantit la reproductibilité
└── README.md
```

Bonus (optionnel)

- **Orchestration du pipeline avec Airflow** : modéliser le pipeline complet (ingestion → prétraitement → clustering → labellisation → classification → tracking MLflow) sous forme de DAG, découpé en tâches indépendantes avec dépendances et fréquence d'exécution définies, gestion des erreurs/retries, et supervision via l'interface Airflow.